

SINGAPORE-MIT ALLIANCE

Computational Engineering
CME5232: Cluster and Grid Computing Technologies for Scientific Computing
COMPUTATIONAL LAB No. 3
19th July, 2006

MPI Implementations on Elementary Algorithms

Objective

To experience MPI Library functions focusing on point to point and collective communications. To experience MPI Library functions focusing on grouping data for communications, communicators and topologies

Background knowledge

C/C++ languages, basic MPI functions and some ideas about communicator and topologies

1. Matrix-Vector Multiplication

Consider a dense square matrix of order n and number of processes p , and assume that n is divisible by p . This matrix is stored in a row-major order in a file. There are several ways to illustrate the matrix-vector multiplication. One way is to block-striped-partition the matrix column-wise, the other way is to block-striped-partition it row-wise, and another way is to block-checkerboard partition it. You are required to develop a parallel program to carry out a matrix-vector multiplication as follows:

(a) Block-striped partitioning – column-wise block-striped partitioning

In this question, we are interested in the column-wise block-striped partitioning.

Write a function that reads in this dense square matrix. Process 0 should read in the number of rows n and broadcast n to all the other processes. Process 0 should then read in a block of n rows and distribute blocks of n/p columns to each of the processes such that the 1st n/p columns go to process 0, the next n/p columns go to process 1 and so on.

Once the distribution of the matrix is done, perform the parallel matrix-vector multiplication and print the result of the matrix-vector multiplication into an output file **result.txt**.

(b) Block-striped partitioning – row-wise block-striped partitioning

Repeat the procedures in (a) for the matrix-vector multiplication using row-wise block-striped partitioning of the matrix.

(c) Block-checkerboard partitioning

Repeat the procedures in (a) for the matrix-vector multiplication using block-checkerboard partitioning of the matrix. State the additional requirements for n and p in order to use block-checkerboard partitioning.

Comment on the differences between the column-wise block-striped partitioning in (a), the row-wise block-striped partitioning in (b) and the block-checkerboard partitioning in (c) in the matrix-vector operation.

Demonstrate all of the above by explaining what you have done, by writing a program, executing the program on the SMA Hydra cluster with the given sample file **matrix_vector_16.txt** of $n=16$ by using $p=4$, and showing the sample outputs.

Measure the timings required to compute the matrix-vector multiplication using the column-wise block-striped partitioning in (a), the row-wise block-striped partitioning in (b) and the block-checkerboard partitioning in (c) in the matrix-vector operation for $n = 16, 32, 64, 128, 256, 512, 1024$ with $p = 4, 16$. Comment on the timings and show the jumpshot diagrams ($n = 256$).

2. Matrix-Matrix Multiplication Using Fox's Algorithm

Study the serial code (**serial_mat_mult.c**) and the parallel implementation of the Fox's algorithm (**fox.c**) for matrix-matrix multiplication involving square matrices A and B. Using sample input file **matrix_matrix_16.txt** containing square matrices A and B of size $n = 16$, run the serial code **serial_mat_mult.c**, and parallel code **fox.c** with $p = 1, 4$ and 16 for the code verification. See the notes attached at the end of this question on how to compile and execute the serial code **serial_mat_mult.c** and parallel code **fox.c**.

Then insert suitable timing measurement code into both serial and parallel codes. Run the serial code **serial_mat_mult.c**, and parallel code **fox.c** with $p = 4, 16$ using square matrices of size $n = 16, 32, 64, 128, 256$ and 512 . Record and compare the timings required for both serial and parallel codes. (Note: **squareMatrixMatrixGenerator.cpp** is provided for you to generate square matrix A and B of different sizes).

When we discussed Fox's algorithm we observed that it would be unrealistic to expect the system to provide n^2 physical processors. So, we modified our original algorithm so that each process stored sub-matrices of order $n/\sqrt{p}$. This modification is implemented in **fox.c**.

Now, our basic algorithm in which we store a single row of the matrix on each process is also unrealistic. Outline a modification to this algorithm so that it stores a block of n/p rows of matrices A and B on each process, and at each stage gathers n/p columns of B onto the process (**fox_row.c**). Compare the storage requirements of the modified original Fox's algorithm (**fox.c**) and modified block of rows Fox's algorithm (**fox_row.c**). Carry out the parallel matrix-matrix multiplication of 2 square matrices. Run **fox_row.c** with $p = 4, 16$ using square matrices of size $n = 16, 32, 64, 128, 256$ and 512 , compare the timing with **fox.c** and show jumpshot diagrams ($n = 256$).

Notes: The following shows how to compile and execute the serial code **serial_mat_mult.c** and parallel code **fox.c**.

(i) Running serial code **serial_mat_mult.c**

```
$ gcc serial_mat_mult.c -o serial_mat_mult
$ ./serial_mat_mult<matrix_matrix_16.txt
```

(ii) Running parallel code **fox.c**

```
$ mpicc fox.c -o fox -lm -mpilog
$ vi fox_sge.sh (Note: Refer to the sample code fox_sge.sh)
$ qsub -pe mpich 4 fox_sge.sh
```

Due date 26th July 2006